

LECTURE 3

Database Languages and Functions

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

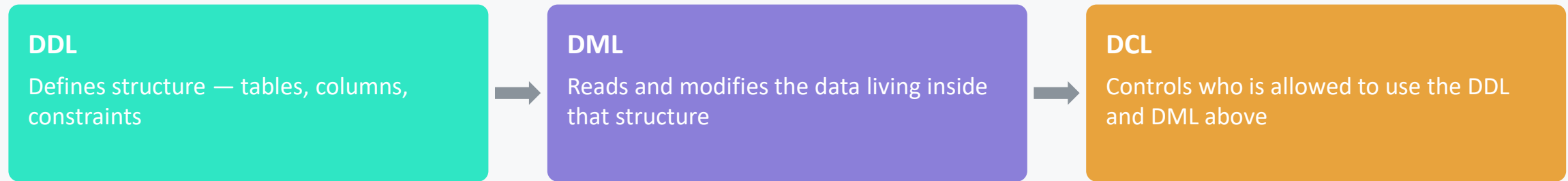
In This Lecture

- Why database languages are split into three sub-languages by purpose
- Data Definition Language (DDL): defining structure
- Data Manipulation Language (DML): working with data, procedural vs. declarative
- Data Control Language (DCL): security and access control
- The complete list of core DBMS functions, expanded from Lecture 2
- Database utilities and the day-to-day tasks of database administration

Why Split the Language by Purpose?

Talking to a DBMS involves three genuinely different kinds of statements: what the data looks like, working with the data itself, and who is allowed to do either.

The three sub-languages of a database language



One language, three jobs. In a modern relational DBMS, DDL, DML, and DCL are usually all part of the same language — SQL — rather than three separate languages learned one at a time. The split here is by purpose, so each part can be reasoned about on its own; SQL itself is covered later in this course, once the relational model is established.

Data Definition Language (DDL)

DDL defines, alters, and removes the database's structure — its schema — rather than the data inside it. For a library system, DDL creates the Book and Member relations, and later adds a column Book didn't originally have.

Book (isbn, title, author, available)

isbn	title	author	available
978-0132350884	Clean Code	Robert C. Martin	true
978-0201633610	Design Patterns	Gamma, Helm, Johnson, Vlissides	false



Create

A new table, view, index, or entire database



Alter

Add, remove, or modify a column; add a constraint



Drop

Remove a table, view, or index — and typically its data

Every DDL statement, when executed, updates the system catalog — which is precisely why DDL changes are felt everywhere at once: every part of the DBMS that consults the catalog immediately sees the new structure.

Building the Library Schema, Step by Step

- Step 1 — CREATE TABLE Book declares isbn, title, author, and available, with isbn as the primary key
- Step 2 — CREATE TABLE Member declares the member's ID, name, and contact details, again with a primary key
- Step 3 — ALTER TABLE Book ADD publicationYear: the library starts tracking a year it never recorded before
- Step 4 — every existing Book row gets a NULL publicationYear until someone fills it in; no data is lost
- Step 5 — the catalog records the new column, so every later query and application sees it immediately

Why DDL Changes Are Felt Everywhere at Once

- Every DDL statement writes to the system catalog — the metadata layer introduced in Lecture 2
- Any component that consults the catalog sees the change at once: the query processor, the optimizer, the authorization checks
- So a column added from one terminal is immediately queryable by the cataloging librarian's reports
- The flip side: a DROP is equally immediate, which is why DDL is restricted to a small set of trusted accounts
- One DDL statement can change what every application sees — treat it as a high-impact operation

Data Manipulation Language (DML)

Where DDL shapes the container, DML works with what's inside it: retrieving existing data and inserting, updating, or deleting rows.

Retrieval

Asking questions of existing data without changing anything

Update

Inserting new data, modifying existing data, or deleting data

Procedural vs. declarative DML — a distinction worth remembering early

Procedural DML

Say what data is needed AND how to get it, step by step (relational algebra)

Declarative DML

Specify only what data is needed; the optimizer decides how — what SQL and relational calculus do

A single logical action — “a member returns Clean Code” — typically involves more than one DML statement working together: marking the book available, and removing the corresponding loan record. Lecture 29's transaction concept guarantees such groups either all succeed or none do.

Retrieval, Step by Step: Which Books Are Available?

- The question: which books by Robert C. Martin are currently available?
- Step 1 — identify the author, Robert C. Martin, and the column that holds it (Book.author)
- Step 2 — keep only rows where author matches and available is true
- Step 3 — return only the columns the question needs, such as title and isbn
- Notice: nothing in the question says how to scan, which index to use, or what order to read rows in
- That gap is exactly what declarative DML hands over to the query optimizer

Procedural vs. Declarative: The Same Question Two Ways

- Procedural (relational-algebra style): take Book, select rows where author is Martin, select rows where available is true, then project title and isbn
- Every step is spelled out in order; the writer of the query controls the evaluation sequence
- Declarative (SQL style): `SELECT title, isbn FROM Book WHERE author = 'Robert C. Martin' AND available = true`
- The declarative version states only the result wanted; the optimizer chooses an index or a full scan
- Both produce the same rows — the difference is who decides how the rows are fetched

Returning a Book: One Action, Several Statements

- The logical action: a member returns Clean Code
- Statement 1 (UPDATE): set Book.available to true for that isbn
- Statement 2 (DELETE): remove the matching loan record
- Neither statement alone describes “the return” — the business event needs both
- If the server fails between them, the book shows as available while the loan still exists, or the reverse
- Lecture 29's transactions guarantee the group either completes together or not at all

Data Control & Security (DCL)

DCL governs permissions: who may run which DDL and DML statements against which parts of the database. Without it, every user with any access would have unrestricted access to all of it.

- Grant — give a specific privilege (read, insert, update, delete, or the right to grant further privileges) to a named user or role
- Revoke — take back a previously granted privilege

DCL in the library system — who can do what to Book

Front-desk clerk

Granted: SELECT, UPDATE (availability only)

Cataloging librarian

Granted: SELECT, INSERT, UPDATE, DELETE

Library patron (self-service)

Granted: SELECT only

DCL is one concrete mechanism behind the “authorization services” function named in Lecture 2 — the DBA uses it to translate an organization's security policy into rules the DBMS actually enforces on every request.

Granting and Revoking Privileges, Step by Step

- Step 1 — GRANT SELECT, UPDATE ON Book TO front_desk_clerk: the clerk can look up and change availability
- Step 2 — GRANT SELECT, INSERT, UPDATE, DELETE ON Book TO cataloging_librarian: full catalog maintenance
- Step 3 — GRANT SELECT ON Book TO patron_portal: read-only access for self-service search
- Step 4 — when a clerk leaves: REVOKE SELECT, UPDATE ON Book FROM front_desk_clerk takes the privilege back
- The policy lives in these statements, and the DBMS enforces it on every request — not only on the ones an application remembers to check

Which Language Does This Task Use?

The same few statement types cover almost every library task. Match each task to the sub-language that owns it.

Task	Language	Example statement
Create the Book table	DDL	CREATE TABLE Book (...)
Add a publication year	DDL	ALTER TABLE Book ADD publicationYear
Find available Martin books	DML (retrieval)	SELECT title FROM Book WHERE ...
Record a returned book	DML (update)	UPDATE Book SET available = true ...
Let a clerk update availability	DCL	GRANT SELECT, UPDATE ON Book TO ...
Retire the old catalog table	DDL	DROP TABLE OldBook

Database Utilities

Lecture 2's function list rounds out with the operational side. Utilities are supporting programs, usually bundled with the DBMS, that help administer the database without being part of the core query engine:

Utility	Purpose	Library-system example
Loading	Bulk-importing data from an external source	Importing 10,000 records from the old system
Unloading / export	Extracting data out to a file	Exporting overdue-loan records for a report
Backup	Making a recoverable copy of the database	Nightly backup of the entire library database
Reorganization	Restructuring physical storage for performance	Rebuilding a fragmented index on Book.isbn
Performance monitoring	Tracking query speed and resource use	Noticing “search by author” has gotten slow
Statistics analysis	Gathering data distribution info for the optimizer	Recording 40% of loans are computer-science books

Utilities in One Library Day

- Morning — a loading job imports 10,000 records from the old catalog system
- Midday — performance monitoring flags that “search by author” has slowed as the catalog grew
- Afternoon — a reorganization rebuilds the fragmented index on Book.isbn to restore speed
- Evening — a statistics run records that 40% of loans are computer-science books, so the optimizer plans better
- Night — a backup captures the whole library database, and an export produces the overdue-loan report
- None of these tools runs itself: someone scheduled each one, which is the next slide's point

Database Administration Functions

Database administration is the ongoing, human side of keeping a database healthy — the day-to-day work of the DBA role:

- **Managing data storage and structure** — physical storage decisions and creating DDL-defined structures
- **Enforcing security and authorization** — issuing and revoking DCL privileges as staff join, change roles, or leave
- **Monitoring performance and tuning** — watching for slow queries and adjusting indexes or configuration
- **Managing backup and recovery** — scheduling backups and rehearsing recovery before a real failure
- **Maintaining data integrity** — reviewing that constraints still reflect the organization's rules
- **Liaising with users** — translating user and developer needs into database changes

A common mix-up: utilities vs. administration. Utilities are tools; administration is the ongoing responsibility that uses those tools. A backup utility can exist for years without protecting anyone if no administrator schedules it to run.

Administration Is a Routine, Not an Event

- Administration runs continuously — not just at go-live or when something breaks
- Weekly: review integrity constraints against the library's current rules
- Monthly: rehearse recovery from last night's backup, so the first real test is not during a real failure
- Ongoing: revoke privileges as staff join, change roles, or leave — stale access is a security hole
- Ongoing: tune indexes and configuration in response to monitoring results
- Always: turn new requests from librarians and developers into schema changes the DDL can deliver

From Languages to Administration: Recap

- We separated database languages by purpose: DDL defines, DML works with data, DCL controls access
- DDL changes the catalog; DML changes rows; DCL changes who may do either
- DML can be procedural (say how) or declarative (say what) — modern SQL is declarative
- Utilities are the tools; administration is the DBA's ongoing responsibility that uses them
- Together these make Lecture 2's functions real: storage, transactions, security, recovery, and more

Key Takeaways

- Database languages split by purpose into DDL (define structure), DML (work with data), and DCL (control access) — usually unified into one language such as SQL
- DDL creates, alters, and drops schema objects; every DDL statement updates the system catalog immediately
- DML retrieves and updates data; it can be procedural (specify how) or declarative (specify only what) — modern SQL is predominantly declarative
- DCL grants and revokes privileges, turning security policy into rules the DBMS enforces on every request
- Database utilities (load, backup, reorganize, monitor) are the tools; database administration is the DBA's ongoing responsibility that relies on them day to day

Looking Ahead: Lecture 4

- Lecture 4 shifts from what you can say to a DBMS to how a DBMS is architected internally
- You'll meet the client-server structure that connects applications to the DBMS
- You'll see the three-schema architecture: external, conceptual, and internal levels
- That architecture is what makes data independence possible — the mechanism behind Lecture 2's promise